

Protótipo e objetivos de arquitetura



Projeto: MultiPower: Aplicação para Condutores de Veículos Elétricos

Grupo: 113985: José Mota
119480: André Silva
119488: Martim Gomes
119758: Fernando Ferreira

Data/versão: 2025/04/16, v2

Sumário:

A Multipower é uma aplicação desenvolvida para facilitar a localização, reserva e pagamento de estações de carregamento para veículos elétricos. A plataforma integra múltiplos fornecedores, permitindo que os utilizadores possam planear rotas otimizadas, comparar preços e receber notificações sobre disponibilidade e promoções. Além disso, são promovidas parcerias B2B, oferecendo benefícios exclusivos e serviços adicionais.

Neste documento está detalhada e explicada a arquitetura do sistema utilizada. Estão ainda referidos os objetivos gerais da mesma, bem como requisitos com impacto na arquitetura, entre outros aspetos. Para além disto, está ainda presente neste documento o protótipo das interações que foi desenvolvido, bem como o resumo do mesmo.

Conteúdo

1 Introdução	3
1.1 Sumário executivo	3
1.2 Controlo de versões	3
1.3 Referências e recursos suplementares	3
2 Arquitetura do sistema	4
2.1 Objetivos gerais	4
2.2 Requisitos com impacto na arquitetura	5
2.3 Decisões e justificação	6
2.4 Arquitetura do software	9
3 Protótipo das interações	12

1 Introdução

1.1 Sumário executivo

Este relatório apresenta a arquitetura proposta para a solução e os resultados da prototipagem, tendo por base o âmbito funcional já caracterizado no relatório de análise de requisitos.

A solução proposta inclui um portal para o cliente, interfaces móveis e web responsivas, um backend modular para fornecedores e operadores, um sistema de APIs para integração com estes e um painel de gestão técnico-administrativo.

Um aspeto essencial no desenho da plataforma computacional é a abordagem “mobile first”, que exige aplicações responsivas e adaptadas a diferentes dispositivos e contextos de uso, garantindo uma experiência consistente e otimizada, independentemente do terminal que for usado.

A prototipagem rápida foca-se nas histórias relativas a concretizar o âmbito do que foi prototipado, assegurando que as funcionalidades testadas estão alinhadas com os épicos definidos. O objetivo é validar, de forma visual e funcional, os principais fluxos da aplicação com base nas necessidades que foram identificadas nas user stories.

1.2 Controlo de versões

Quando?	Responsável	Alterações significativas
8/04/2025	André Silva Martim Gomes Fernando F. Francisco Mota	Início do o relatório
12/04/2025	Martim Gomes Fernando F. Francisco	Continuação do relatório Início da Realização do Protótipo
13/04/2025	Martim Gomes Fernando F. Francisco Mota André Silva	Conclusão do Protótipo Realização da apresentação
16/04/2025	Fernando F. (25%) Martim Gomes (25%) Francisco Mota (25%) André Silva (25%)	Integração no relatório da parte do protótipo

1.3 Referências e recursos suplementares

Na parte da prototipagem procuramos tentar tirar ideias de outros sites. Um exemplo disso é, por exemplo, para criar conta/dar login onde nos tentamos inspirar no Facebook. Para além disso, tentamos buscar referências de outros sites sobre veículos elétricos, como por exemplo, o site da Tesla.

2 Arquitetura do sistema

2.1 Objetivos gerais

Os objetivos gerais para a MultiPower são:

- Os utilizadores devem poder aceder à app em qualquer plataforma (Android/Android auto, IOS, android, etc.).
- Os condutores e fornecedores devem ter acesso aos dados relevantes das estações de carregamento em tempo quase real, usando uma visualização de mapa da cidade, com pop-ups em cada estação de interesse. O seguimento do Conductor deve ser feito a partir do GPS do dispositivo que acede à aplicação.
- A carteira de pagamentos deve ser segura com identificação a dois fatores, e universal para todos os distribuidores presentes nas estações da aplicação.
- Comunicação entre condutor-fornecedor e utilizador-equipa técnica eficaz e rápida. Isso inclui a geração automática de tickets, notificações e atualizações sobre a resolução dos problemas, garantindo a transparência e o acompanhamento contínuo do processo de correção.
- Os clientes registados devem receber notificações (*push notifications*) a anunciar novas oportunidades. Para isso, é importante que haja uma adesão generalizada ao uso das aplicações móveis, na plataforma Android e iOS.
- Interface intuitiva para planeamento de rotas, relato de problemas, adesão ao plano premium e reserva de pontos de carregamento e oportunidades, facilitando a interação do utilizador com o sistema e garantindo respostas rápidas e relevantes em tempo real.
- Eficiência no modo offline da aplicação, o acesso a rotas pré definidas e informações baseadas em normas(hora do dia, fluxo normal de movimento, etc) deve ser rápido e correto.

2.2 Requisitos com impacto na arquitetura

Na tabela abaixo, encontra-se uma listagem de requisitos que influenciam diretamente o design da arquitetura::

Requisitos	Descrição
RD - 1	O sistema deve suportar a utilização sustentada de 25000 sessões em simultâneo, de forma a garantir um programa totalmente funcional, mesmo durante possíveis picos de utilização.
RD - 2	A aplicação deve carregar a lista de estações de carregamento em menos de 2 segundos, em condições normais de rede.
RD - 3	Todos os serviços que suportam o sistema devem ser configurados de forma redundante, com recurso a um ambiente em cloud, para assegurar uma disponibilidade de 99.9%
RS - 1	Todos os utilizadores devem autenticar-se utilizando um método seguro, como por exemplo, autenticação de dois fatores para efetuar pagamentos, de forma a proteger contra acessos não autorizados.
RS - 2	Os dados de transações de carregamentos devem ser encriptados antes de serem armazenados no sistema, focando-se numa segurança pós-transação.
RU - 1	Um novo utilizador deve conseguir entender e utilizar a aplicação sem necessidade de um tutorial extenso.
RU - 2	Todas as mensagens de erro devem ser apresentadas de forma clara, com sugestões de resolução quando necessário.
RU - 3	O portal da loja deve-se ajustar para ter uma apresentação adequada ao ecrã, designadamente para smartphones, tablets ou sistemas de secretária.
RINT - 1	A aplicação deve integrar-se com sistemas de pagamentos externos de forma transparente e sem necessidade de intervenção manual.
RINT - 2	A aplicação deve integrar-se com APIs externas.
RTF - 1	Se uma estação de carregamento estiver offline, a app deve informar os utilizadores e sugerir alternativas próximas
RTF - 2	O sistema deve garantir que uma transação de pagamento incompleta seja revertida automaticamente para evitar cobranças indevidas

Cada um dos requisitos mencionados ao longo da tabela, contribuem para influenciar a arquitetura do nosso sistema. Ao nível dos requisitos de desempenho, todos os que foram apontados, exigem uma infraestrutura robusta e escalável (por exemplo: existe um aumento significativo de acessos à aplicação). Já os requisitos de segurança estão direcionados para a inclusão de serviços dedicados à proteção de dados e ao controlo de acesso. Para além disso, os requisitos de usabilidade remetem-se para o fácil entendimento das diferentes funcionalidades, assim como, ao nível da acessibilidade entre os variados, e diferentes dispositivos. Os requisitos de interface complementam este grupo, focando-se na integração com sistemas externos, como APIs. Por fim, os requisitos de tolerância e falhas tentam garantir que o sistema seja resiliente e confiável.

Ao atender a cada uma das necessidades, a aplicação MultiPower tenta, assim, garantir aos seus utilizadores, uma experiência satisfatória e confiável.

2.3 Decisões e justificação

Tendo em conta os objetivos para a arquitetura, e os requisitos levantados na Análise, foram tomadas as seguintes decisões:

1. Tecnologias de FrontEnd:

Decisão: Implementar a camada de interface utilizando React(para Web) e React Native (para dispositivos móveis).

Justificação:

- A equipa de desenvolvimento já possuía experiência prévia com a biblioteca React, o que diminui a curva de aprendizagem, permite poupar tempo e dinheiro e potencia o desenvolvimento iterativo do sistema.
- A transição para Reactive Native permite reutilizar componentes usados na biblioteca React, reduzindo assim o tempo de implementação das apps móveis.
- As duas bibliotecas contam com uma comunidade ativa e vasta documentação publicada, facilitando a correção de erros e integração de bibliotecas externas.

2. Arquitetura de BackEnd e Infraestrutura:

Decisão: Adotar uma arquitetura com Node.js e Express.

Justificação:

- Node.js oferece uma boa performance e escalabilidade para aplicações que exigem manipulação de sockets ou streams em tempo real (por exemplo, monitorização de carga do veículo)
- A equipa de desenvolvimento tem experiência prévia em JavaScripts/ TypeScript, facilitando a manutenção e permitindo partilhar parte da lógica da validação entre BackEnd e FrontEnd.
- Express é uma framework minimalista e bastante difundida, garantindo flexibilidade para integração futuras.

3. Framework de Mapas Digitais:

Decisão: Usar Mapbox como plataforma de mapas

Justificação:

- O Mapbox disponibiliza um plano gratuito e com menos limitações do que outras opções como o Google Maps. Além disto não são impostas marcas d'água melhorando a experiência visual na utilização da aplicação.
- Oferece SDKs específicos para para Android, IOS e Web, agilizando a integração e tornando as experiências consistentes nos mais diversos dispositivos.
- É ainda possível personalizar o estilo dos mapas, mantendo a identidade visual da MultiPower. Sendo assim é possível adicionar os postos de carregamento com o estilo que for pretendido, não tendo limitações.

4. Base de Dados

Decisão: Utilizar uma base de dados SQL / NoSQL

Justificação:

- (SQL): O modelo relacional é robusto e pode lidar bem com a principal necessidade de registrar transações (pagamentos,reservas, etc) de forma consistente.
- (NoSQL): Se a aplicação tiver que escalar rapidamente a leitura de dados sobre estações e preferir esquemas mais flexíveis (por exemplo, armazenar dados de rotas em tempo real).

5. Processamento de Transações em Lote:

Decisão: Agrupar e enviar transações aos parceiros(fornecedores de estações de carregamento e plataformas de pagamento) num lote a cada X minutos (por exemplo 5 minutos).

Justificação:

- Reduz a sobrecarga de chamadas contínuas aos parceiros, que podem ter limites de rate ou custos de API.
- Facilita o registo de auditoria das transações: as transações ficam registadas internamente, e só depois de confirmadas num lote, são efetivamente atualizadas no sistema externo.
- Oferece um balanço entre consistência e desempenho, pois as confirmações para o utilizador podem ser quase imediatas na nossa app, mas a comunicação de final de cada lote com os parceiros é feita de forma assíncrona.

6. Gestão de Pagamentos e Segurança:

Decisão: Integrar com Payment Gateway externos(como por exemplo, PayPal, MBWay, cartões, etc) para processar pagamentos.

Justificação:

- Evitar a necessidade de lidar diretamente com dados de cartões bancários, reduzindo os riscos de segurança e problemas de conformidade PCI-DSS(conjunto de normas de segurança que protege os dados dos cartões de pagamento)
- Estes gateways têm APIs simples de integrar e oferecem funcionalidades como deteção de fraude, gestão de reembolsos e conformidade com regras de cada país.
- Facilita a entrada em novos mercados, permitindo realizar pagamentos com as moedas locais.

7. Notificações em Tempo Real:

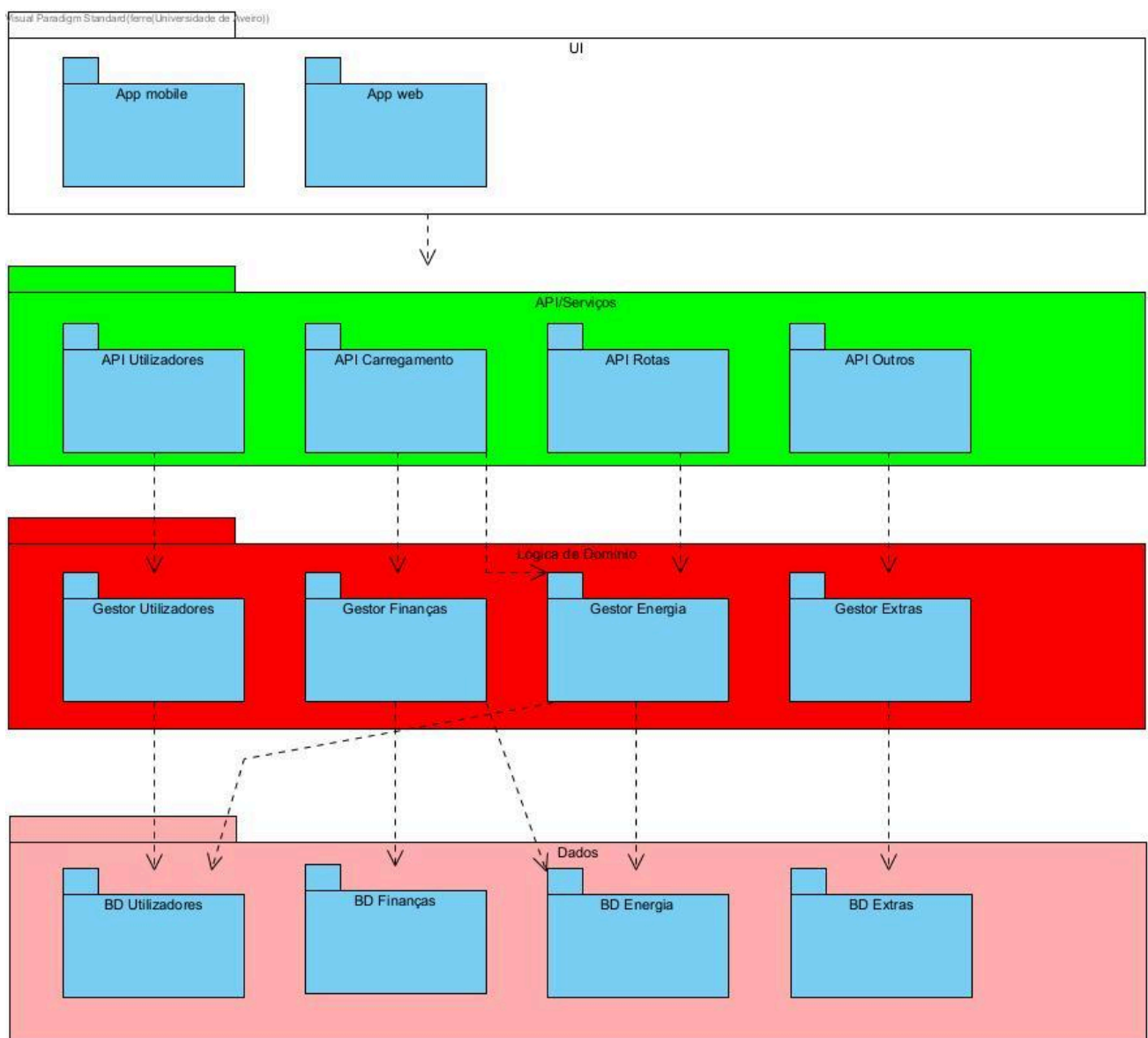
Decisão: Utilizar um serviço de mensagens (Firebase Cloud Messaging) para enviar notificações push.

Justificação:

- A necessidade de alertas em tempo real sobre reserva, promoções, finalização do carregamento, etc. exige um sistema de notificações escalável.

- A Firebase já fornece integrações simplificadas com apps IOS, Android, e um método robusto para envio segmentado de mensagens.
- O custo é relativamente acessível para o volume esperado e a implementação é bem documentada.

2.4 Arquitetura do software



A arquitetura do MultiPower apresenta uma abordagem que garante que os diferentes domínios do sistema (gestão de utilizadores, sessões de carregamento, rotas, finanças, entre outros) estão devidamente separados em serviços independentes e escaláveis. Esta divisão permite uma elevada escalabilidade seletiva e facilidade na manutenção e futura evolução da plataforma.

O modelo apresentado no diagrama segue assim uma organização feita por camadas, na qual cada um dos diferentes domínios está estruturado em camadas bem definidas, respeitando o princípio da separação de responsabilidades. Assim, é possível garantir uma arquitetura clara, modular e de fácil evolução, na qual a colaboração entre os módulos ocorre de forma hierárquica visto que cada camada interage apenas com a que se encontra imediatamente abaixo.

Cada camada do diagrama tem um papel bem definido e comunica apenas com a camada que está imediatamente abaixo. Por exemplo, as interfaces do utilizador não interagem diretamente com a lógica de domínio, mas sim através das APIs, que atuam como pontos de controlo e adaptação entre o que o utilizador pretende fazer e as regras que o sistema impõe. Além disso, esta arquitetura fornece uma base sólida para futuras evoluções que possam vir a existir no sistema. Por estar assente numa lógica clara de camadas e módulos, torna-se mais simples introduzir novas funcionalidades, integrar serviços externos ou até substituir certos componentes internos por versões mais modernizadas, sem nunca comprometer o funcionamento global da plataforma.

A primeira camada, correspondente à interface do utilizador (UI) e representa a aplicação visível ao utilizador, seja através da aplicação mobile ou através da aplicação web. Esta camada é responsável por apresentar a informação relevante ao utilizador e por recolher as suas interações, tais como iniciar uma sessão de carregamento, consultar trajetos, aceder ao histórico de sessões, ou usufruir de promoções e campanhas que estejam em vigor em vigor. Assim, sempre que o utilizador realiza alguma ação, esta é traduzida num pedido que é enviado para as APIs da camada seguinte. Por exemplo, quando um utilizador pretende fazer login ou editar o seu perfil, a UI comunica com a API Utilizadores, que por sua vez trata da autenticação e da atualização dos dados do perfil. Se o utilizador já quiser iniciar o carregamento de um veículo, a UI comunica com a API Carregamento, que vai validar se os critérios estão de acordo com o esperado para começar essa operação. Por outro lado, se pretender planear uma rota para uma viagem, introduz o destino na aplicação e a UI envia esta informação à API Rotas, que vai responder com um trajeto otimizado, tendo em conta fatores importantes como os pontos de carregamento disponíveis e o estado da bateria. Por fim, para ações relacionadas com notificações, campanhas promocionais, histórico de sessões ou outros serviços mais complementares, a UI comunica com a API Outros, que é responsável por centralizar esse tipo de funcionalidades adicionais.

Avançando para a segunda camada, a camada de serviços/API é responsável por receber os pedidos provenientes da interface e encaminhá-los para os componentes adequados presentes na lógica de domínio. Esta camada atua como intermediária entre o que o utilizador deseja fazer e as regras que o sistema deve respeitar. Observando o diagrama, é então possível perceber as relações entre cada API e os respetivos gestores da lógica de domínio. A API Utilizadores depende diretamente do Gestor Utilizadores, sendo este responsável por verificar credenciais, aplicar validações de dados e também gerir o ciclo de vida que um utilizador possui dentro da plataforma. A API Carregamento possui dependência tanto do Gestor Energia como do Gestor Finanças, uma vez que o início de uma sessão de carregamento está condicionado à validação de pagamento,

que é realizada pelo Gestor Finanças, e também à gestão do estado da sessão de carregamento, que é feita pelo Gestor Energia. A API Rotas depende do Gestor Energia, uma vez que é este que tem a lógica necessária para processar dados de localização, calcular trajetos e também definir os pontos considerados ideais para as paragens para carregamento. Por último, a API Outros depende diretamente do Gestor Extras, que agrega diversas funcionalidades extras, tais como a gestão de notificações, promoções ou até de dados históricos de carregamento.

A terceira camada é a camada da lógica de domínio e constitui o núcleo funcional da aplicação. É nesta camada que se encontram as regras de negócio principais, bem como o comportamento da aplicação perante diversos pedidos e eventos. Cada gestor desta camada opera com extrema autonomia sobre o seu domínio específico, comunicando apenas com os módulos de dados que lhe dizem respeito. O Gestor Utilizadores é assim responsável por gerir perfis, autenticação e dados pessoais dos utilizadores, recorrendo também à base de dados BD Utilizadores com o objetivo de validar essa informação. O Gestor Finanças trata das operações financeiras, tais como verificação de saldo, registo de transações, atualização de carteiras virtuais e também aplicação de descontos, sendo estas operações realizadas através do acesso à BD Finanças. O Gestor Energia gere todas as sessões de carregamento, monitoriza o estado dos postos, calcula estimativas de duração e pode, quando é estritamente necessário, cruzar informações com a BD Energia para assegurar que tudo decorre como estava previsto. O Gestor Extras reúne um conjunto de funcionalidades adicionais que enriquecem a experiência do utilizador na aplicação, como é o caso da gestão de notificações, das campanhas promocionais e do registo histórico de atividades. É através deste módulo que se processam, por exemplo, os alertas que são posteriormente enviados ao utilizador, as condições de aplicação de promoções que estão em vigor ou até o acesso a dados anteriores que estejam relacionados com carregamentos. Para isso, comunica com a BD Extras, onde consulta, atualiza ou remove os dados relevantes, consoante o que necessita naquele momento.

A quarta e última camada é a camada de dados, onde se encontram armazenadas todas as informações essenciais ao funcionamento da aplicação. Esta camada tem como principal responsabilidade fornecer dados persistentes à lógica de domínio, garantindo que esta toma decisões com base em dados, estando estes dados devidamente atualizados e sendo também eles fiáveis. Assim, a BD Utilizadores contém informações relacionadas com contas, perfis e credenciais, a BD Finanças armazena dados sobre pagamentos, carteiras virtuais e histórico de transações, a BD Energia regista as sessões de carregamento, o estado das estações e respetiva localização e a BD Extras guarda notificações que são emitidas, campanhas promocionais, histórico de utilização de serviços e também dados adicionais que não se enquadram nos outros domínios. Desta forma, a arquitetura do MultiPower mantém-se flexível e com responsabilidades bem definidas, o que facilita não só a sua manutenção como também o crescimento futuro que esta plataforma possa vir a ter.

3 Protótipo das interações

A interação proposta no protótipo pode ser experimentada no seguinte link (Link do recurso Online(Figma)):

<https://www.figma.com/proto/Kgzi9pNQW9PCF285779J2X/Prot%C3%B3tipo?node-id=1-1120&p=f&t=doNFAAwdyKbLOtsg-1&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=1%3A1120>

Primeiro épico : Gestão de reservas e carregamentos

O primeiro épico que pode depois ser experimentado no link acima encontra-se subdividido em 3 diferentes User Storys. A User Story 1 é a notificação de reserva (Como trabalhador da PowerCharge Lisboa, quero receber uma notificação quando uma nova reserva for feita, para poder agir com rapidez). A User Story 2 é a confirmação de reserva (Como trabalhador da PowerCharge Lisboa, quero poder confirmar que está tudo ok com a reserva feita). Por fim, a User Story 3 é a consulta de estatísticas (Como trabalhador da Powercharge Lisboa quero poder consultar as estatísticas dos últimos carregamentos para acompanhar o estado das estações). Para clarificar, PowerCharge Lisboa é uma empresa que fornece estações de carregamento de veículos elétricos.

Partindo agora para o resumo do fluxo de acontecimentos que pode ser experimentado no link acima, o utilizador da nossa aplicação Multipower começa por entrar no site e vai escolher entre fornecedor ou cliente. Neste caso, foi usado o exemplo de Vicente que é um trabalhador da Powercharge Lisboa. Assim, Vicente ao entrar no site da MultiPower escolhe a opção fornecedor e vai ter de dar login na sua conta, ou caso não tenha conta ainda vai ter de criar uma e só depois dar login. Posteriormente, após o login ter sido corretamente efetuado, na primeira página tem à esquerda a lista de reservas que foram efetuadas no dia atual, bem como notificações que possam vir a surgir, entre outras coisas. Passado algum tempo de Vicente entrar no site, Vicente depara-se que recebeu uma notificação. Ao abri-la, Vicente vê que foi efetuado um carregamento usando uma das estações fornecidas pela PowerCharge Lisboa, logo ele vai confirmar se está tudo bem com a reserva feita. Assim, ele foi levado para uma página com detalhes da reserva, e ao verificar que estava tudo Ok, este voltou para a página inicial, recebendo uma notificação de que a reserva foi verificada com sucesso. Por fim, Vicente decide ainda verificar algumas das estatísticas dos últimos carregamentos de modo a inteirar-se de certos fatores relacionados com os mesmos carregamentos.

Segundo épico : Planeamento de Rota Otimizada

O Segundo Épico é o do Planeamento de Rota Otimizada pelo Cliente. Aqui, o Ricardo, que está a usar a app da MultiPower, pretende realizar uma viagem de Aveiro até ao Porto, mas percebe que a bateria do seu automóvel tem apenas 27%, o que é insuficiente para completar o trajeto completo. Ao clicar no botão de “Definir rota”, encontra uma página, que ao inserir a origem e o destino que pretende, irá lhe ser fornecido um lote de opções de trajetos. Com isto, selecione a opção que permite escolher a melhor rota incluindo um local de carregamento ideal ao longo do percurso.

Com isso, a aplicação MultiPower processa as informações e apresenta automaticamente uma rota otimizada, identificando um ponto de carregamento conveniente para carregar o seu veículo elétrico durante a viagem. Além disso, o Ricardo tem a possibilidade de consultar informações detalhadas sobre o posto de carregamento, como tempo estimado de recarga e preços.

Com esta funcionalidade, o Ricardo garante que a viagem decorra sem preocupações ou atrasos inesperados, tendo um plano claro e confiável que otimiza a eficiência e a conveniência do trajeto.

Terceiro épico : Carregamento da Carteira

O Terceiro Épico é o carregamento da Carteira por parte do Cliente. Aqui, o Ricardo, que é um cliente que está a usar a app da Multipower pela primeira vez, pretende carregar fundos para posteriormente poder pagar os seus carregamentos. Quando clica no botão da carteira, depara-se com duas opções: “Carregamento”, em que se vai carregar a carteira com fundos que vão servir para pagar futuros carregamentos nos postos, e “Pagamento de carregamento do Automóvel” em que o utilizador pode, depois de efetuar o carregamento, pagar diretamente pelo mesmo, sem ter carregado anteriormente a sua carteira. Neste caso, o Ricardo optou pela primeira solução, para evitar possíveis constrangimentos a meio da sua rota já planeada. Com isso, depara-se com 4 meios de pagamento (associar cartão, MB Way, PayPal e Transferência bancária). Aqui o Ricardo escolhe o método “associar cartão”, ao qual depois de inserir os dados, o cartão é então adicionado e o saldo da carteira virtual é atualizado de acordo com o saldo que o Ricardo pretende adicionar. Com este método, o Ricardo poderá assegurar os pagamentos de futuros carregamentos, sem ter de adicionar um novo método, bastando só atualizar o saldo MultiPower.